Appendix

A Detailed Results of RQ1

Table 6. Detailed results of RQ1, where P and R denote precision and recall, respectively, Python and Multilingual denote SWE-TQ-Python and SWE-TQ-Multilingual, respectively, and all metrics are reported in percentage.

		Method	Qwen3.8-27B				DeepSeek-V4-Flash				GPT-5.6 Luna			
			F1↑	P↑	R↑	FPR↓	F1↑	P↑	R↑	FPR↓	F1↑	P↑	R↑	FPR↓
Task Clarity	Python	Vanilla	50.0	**100.0**	33.3	**0.0**	24.7	16.4	50.0	16.3	45.5	38.5	55.6	5.7
		SimpleRAG	43.5	**100.0**	27.8	**0.0**	19.6	11.3	72.2	36.2	48.7	47.4	50.0	3.6
		SPICE	52.8	40.0	**77.8**	7.4	39.6	24.7	**100.0**	19.5	30.8	18.2	**100.0**	28.7
		Codex	20.0	**100.0**	11.1	**0.0**	30.3	33.3	27.8	3.6	52.6	50.0	55.6	3.6
		Claude Code	20.0	**100.0**	11.1	**0.0**	37.2	32.0	44.4	6.0	47.6	41.7	55.6	5.0
		ExLabel	**64.7**	68.8	61.1	1.8	**51.9**	**77.8**	38.9	**0.7**	**61.1**	**61.1**	61.1	**2.5**
	Multilingual	Vanilla	26.7	23.5	30.8	4.8	15.0	8.7	53.9	26.9	17.3	10.5	50.0	20.3
		SimpleRAG	26.9	26.9	26.9	3.5	12.2	6.6	**76.9**	51.7	20.8	14.3	38.5	11.0
		SPICE	13.9	8.3	**42.3**	22.1	15.4	8.7	65.4	32.5	13.2	7.3	**73.1**	44.4
		Codex	17.1	33.3	11.5	1.1	16.6	10.1	46.2	19.6	14.5	9.2	34.6	16.3
		Claude Code	24.2	**57.1**	15.4	**0.6**	16.2	11.0	30.8	11.9	13.3	8.0	38.5	21.0
		ExLabel	**37.5**	40.9	34.6	2.4	**25.4**	**20.0**	34.6	**6.6**	**26.9**	**22.0**	34.6	**5.9**
Test Coverage	Python	Vanilla	10.0	75.0	5.4	0.9	13.3	**100.0**	7.1	**0.0**	12.7	57.1	7.1	**2.8**
		SimpleRAG	66.7	**93.6**	51.8	1.8	31.9	84.6	19.6	1.8	20.9	63.6	12.5	3.7
		SPICE	71.3	63.0	**82.1**	24.8	37.5	62.5	26.8	8.3	75.8	65.8	89.3	23.9
		Codex	10.0	75.0	5.4	0.9	16.9	40.0	10.7	8.3	43.6	**77.3**	30.4	4.6
		Claude Code	0.0	0.0	0.0	**0.0**	9.2	33.3	5.4	5.5	30.6	68.8	19.6	4.6
		ExLabel	**76.5**	84.8	69.6	6.4	**73.0**	71.2	**75.0**	15.6	**79.4**	69.3	**92.9**	21.1
	Multilingual	Vanilla	24.4	50.0	16.1	3.5	33.7	**43.6**	27.4	7.7	20.3	47.1	12.9	**3.2**
		SimpleRAG	26.2	26.7	25.8	15.4	13.3	38.5	8.1	**2.8**	36.0	**47.4**	29.0	7.0
		SPICE	31.7	22.9	**51.6**	37.8	27.0	24.1	**30.7**	21.0	35.7	24.0	**69.4**	47.6
		Codex	14.1	55.6	8.1	**1.4**	19.1	23.3	16.1	11.5	38.9	45.7	33.9	8.7
		Claude Code	16.7	**60.0**	9.7	**1.4**	8.9	14.3	6.5	8.4	35.2	41.3	30.7	9.4
		ExLabel	**36.9**	46.3	30.7	7.7	**34.9**	40.4	**30.7**	9.8	**40.0**	30.1	59.7	30.1

In addition to F1 and FPR, Table 6 reports the precision and recall of all techniques in RQ1.

Table 6 further reveals different precision-recall trade-offs among the techniques. Vanilla, SimpleRAG, Codex, and Claude Code tend to be conservative, and their average recall is only 22.3%–36.6% across the 12 combinations of tasks, subsets, and backbone models. In all 17 cases where one of these baselines achieves a precision no lower than that of ExLabel, its recall is lower than that of ExLabel, such as the precision of 100.0% with a recall of only 11.1%–33.3% on task clarity assessment in SWE-TQ-Python with Qwen3.8-27B. Such conservative predictions leave most low-quality instances in the dataset. In contrast, SPICE achieves the highest recall in 9 of the 12 combinations with an average recall of 67.4%, but its average precision is only 30.8%, which is the lowest among all techniques. For example, SPICE achieves a recall of 73.1% but a precision of only 7.3% on task clarity assessment in SWE-TQ-Multilingual with GPT-5.6 Luna, indicating that most of the instances it flags are valid ones. ExLabel strikes a better balance between the two, achieving the highest average precision of 52.7% and an average recall of 52.0%, which is second only to SPICE. The average gap between the precision and recall of ExLabel is 14.8 percentage points, compared with 29.4–42.5 percentage points for the baselines. Meanwhile, the trade-off

of ExLabel varies with the backbone model. With GPT-5.6 Luna, ExLabel favors recall on test coverage assessment, achieving recalls of 92.9% and 59.7% with precisions of 69.3% and 30.1% in SWE-TQ-Python and SWE-TQ-Multilingual, respectively, which explains its relatively high FPR in RQ1. Overall, this balance enables ExLabel to remove more low-quality instances than the conservative techniques while discarding fewer valid instances than SPICE, which is consistent with its downstream benefits in RQ3.

B Impact of Historical Data Size

Table 7. F1 of ExLabel with GPT-5.6 Luna using different proportions of the historical data for experience accumulation.

Task	Subset	0%	20%	40%	60%	80%	100%
Task Clarity	Python	45.5	**66.7**	64.7	56.3	57.1	61.1
	Multilingual	17.3	20.0	23.2	15.9	22.0	**26.9**
Test Coverage	Python	12.7	**80.0**	77.5	77.9	79.7	79.4
	Multilingual	20.3	38.9	37.3	39.1	39.1	**40.0**

To investigate how much annotated data is required for the experience bank to become effective, we accumulate experience on 20%, 40%, 60%, and 80% of the historical data for each task and evaluate ExLabel with GPT-5.6 Luna on the same test data as RQ1. As shown in Table 7, accumulating experience on 20% of the historical data, *i.e.,* about 400 annotated instances, improves F1 by 2.7–67.3 percentage points over w/o Experience Bank in all four combinations of tasks and subsets. On task clarity assessment, the performance of ExLabel tends to stabilize early, as F1 shows no clear increase beyond 20% of the historical data. This is likely because underspecified instances account for only 4.9% of the historical data, and their typical causes, such as missing key information in the task description, mainly concern the description itself, so the experience bank can capture them from a small amount of data while additional data may contribute limited new experience. On test coverage assessment, F1 shows an increasing trend from 40% to 100% of the historical data, rising from 77.5% to 79.4% and from 37.3% to 40.0% in SWE-TQ-Python and SWE-TQ-Multilingual, respectively. This is likely because inadequate instances account for 32.1% of the historical data, and their causes, such as test patches relying on specific error messages or implementation details, depend on the golden patch, the test patch, and the evolving state of the codebase, which tend to vary across repositories, so additional data may still contribute new experience.

C Training Recipe of RQ3

In RQ3, we perform full-parameter fine-tuning on Qwen3.5-9B-Base [44] using the LLaMA-Factory [63] framework on 4 H100 GPUs. Following prior work [40, 53, 58], we set the batch size to 8, the learning rate to 5×10^{-5}, the maximum number of epochs to 3, and the maximum context length to 131,072 tokens, use a cosine learning-rate scheduler, and set the warmup ratio to 0.1.

D Case Study for Ablation

We present an additional representative case for test coverage assessment in RQ2, as shown in Figure 7. In this case, the test patch depends on a specific error string implemented in the golden patch, namely "Can only compare SkyCoord to Frame with data", which is not explicitly stated in the task description. For **w/o Reranking**, the retrieved experience emphasizes the equivalence between the problematic method and a known-working alternative. This shifts the LLM's focus away from dependence on a specific implementation toward verifying behavioral consistency and

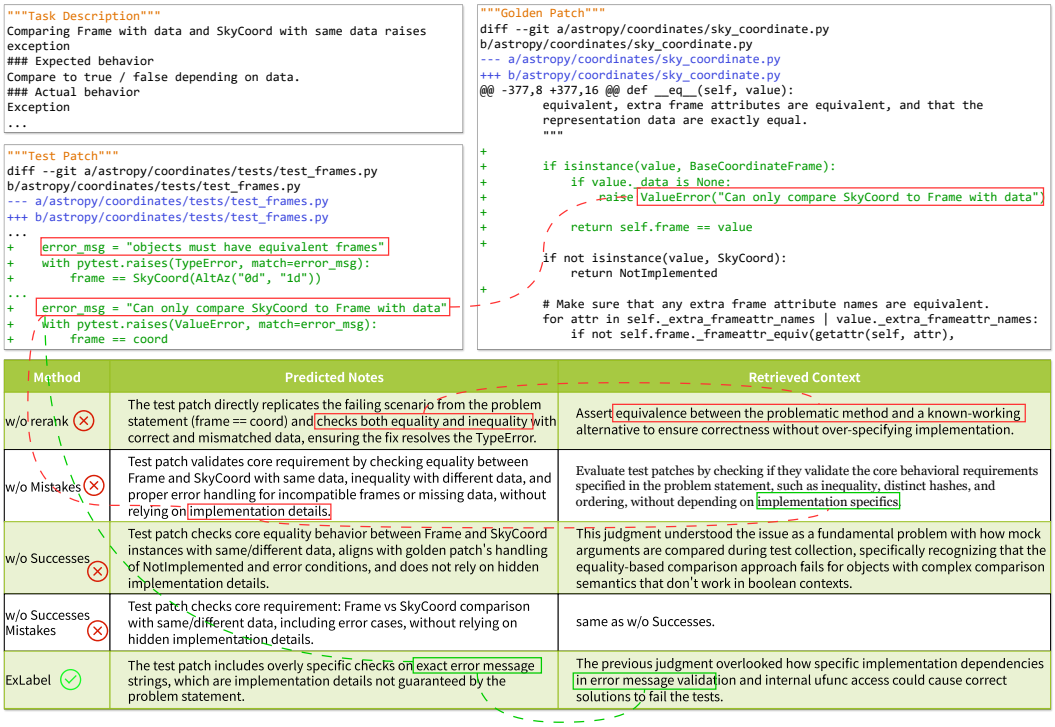


Fig. 7. A test coverage assessment case of `astropy-13477` to reveal the effectiveness of each component in ExLabel.

semantic correctness. For **w/o Field (Mistakes)**, the retrieved experience focuses on the core behavior requirement. Although the experience mentions implementation details, it lacks further elaboration. As a result, the LLM directly concludes that there is no dependency on implementation details after checking the core requirement. For **w/o Field (Successes)** and **w/o Field (Successes, Mistakes)**, both settings retrieve identical experience content, which is an understanding of an unrelated issue about how mock arguments are compared during test collection. This content lacks direct guidance for the current instance. Consequently, the LLM fails to identify the dependency on a specific error message and instead makes a judgment solely based on core behavior. In contrast, ExLabel retrieves experience whose **Mistakes** field explains that a previous judgment overlooked how implementation dependencies in error message validation could cause correct solutions to fail the tests. This experience enables the model to identify the hidden dependency and generate the correct prediction successfully.